LeverageSHAP: A Short, Honest Discussion

What LeverageSHAP is, in one paragraph

`LeverageSHAP` in `leverageshap.py` is a faithful implementation of Algorithm 1 from Musco & Witter (2024). It solves the same weighted least-squares problem as Kernel SHAP, but instead of sampling coalitions with probability proportional to the *Shapley kernel* weight $w(|S|) = \frac{1}{\binom{n}{|S|} \cdot |S| \cdot (n - |S|)}$, it samples them with probability proportional to their *statistical leverage scores* on the underlying design matrix. The paper proves (and this is really the whole point) that those leverage scores have a beautifully simple closed form: $\ell_z = \frac{1}{\binom{n}{|z|}}$. Coalition sizes then receive *equal total weight*, rather than the strong end-of-spectrum emphasis Kernel SHAP puts on very small and very large subsets:

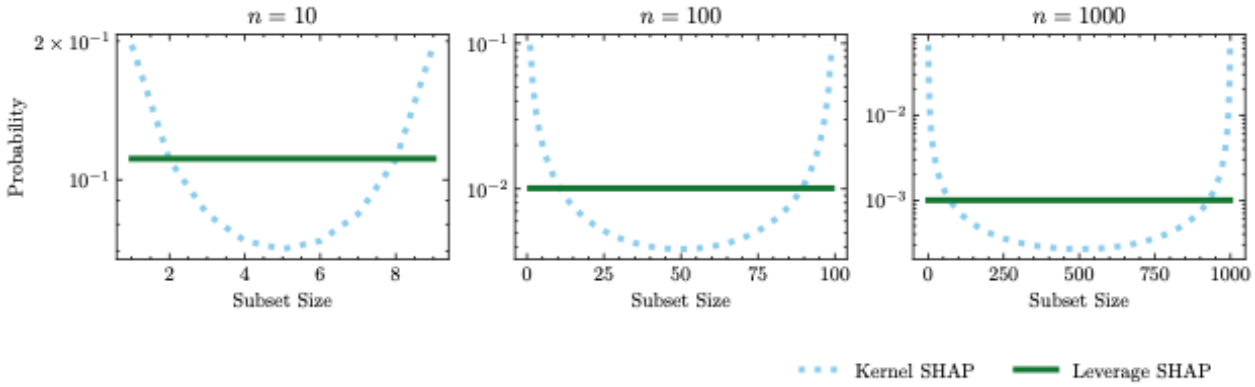

Figure 1: Kernel-SHAP vs leverage-score sampling distributions (Musco & Witter, Fig. 2).

Paired sampling and sampling-without-replacement (via a Binomial-per-size draw plus lexicographic combinatorial unranking, exactly as in Algorithm 2 of the paper) are folded in on top. The theoretical prize is a $(1+\varepsilon)$-approximation guarantee to the *regression objective* using only $m = O(n \log n + \frac{n}{\varepsilon\delta})$ model evaluations in expectation (Theorem 1 in the arXiv version), plus a matching corollary on the ℓ_2-norm error scaled by a problem-specific γ (Corollary 3.2 in the paper).

Where LeverageSHAP wins, where it loses (what the paper predicts)

Before turning to what the benchmark says, it is worth being precise about what the paper actually promises. Reading Section 5 and Appendix A carefully suggests five hypotheses:

1. **Small budgets ($m \ll 2^n$) is exactly where the leverage-score story applies.** The whole $(1+\varepsilon)$ guarantee is proved for the subsampled regression regime, so this is the natural regime to see a gap in favour of LeverageSHAP.
2. **When $m \to 2^n$, both LeverageSHAP and Optimized Kernel SHAP should collapse onto the exact solution.** The paper flags this explicitly: because both methods sample without replacement, they eventually enumerate the full support and hit machine precision. Any ordering between them in that regime is just noise from the linear solve.
3. **Larger n should help LeverageSHAP the most in relative terms.** The analysis is $O(n \log n)$ in m, so as n grows the fixed-budget window where the theory bites shifts *away* from full enumeration and *into* the practically interesting regime. That is precisely where the paper reports its biggest gains (Figure 3 in the arXiv version).
4. **The theorem is about the regression objective, not the ℓ_2-norm of the estimates.** The paper reduces one to the other only via Corollary 3.2, and the reduction picks up a factor $\gamma = \|A\varphi - b\frac{\|}{\|}^2 A\varphi\|^2$. If a game's value function is far from the column span of the design matrix, γ is large and even a perfect objective-value guarantee degrades on the ℓ_2 metric. The paper

warns about this explicitly and even shows error growing with γ for all three regression-based methods (their Figure 5).

5. **Under paired sampling, $\|z\| = 0$ and $\|z\| = n$ never appear in the regression rows.** They enter only through the efficiency shift (line 114 of `leverageshap.py`). That is baked into the algorithm design and is not a source of error.

In short, the expectation is that LeverageSHAP should win in the intermediate m regime, *tie* Optimized Kernel SHAP once m gets close to 2^n, and be *most helpful* on the harder large-n datasets. On tasks where γ is large, none of the three regression-based methods should be expected to perform well on the ℓ_2 metric.

What the benchmark actually shows

Running

```
uv run python -m benchmark.performance \
    --methods OptimizedKernelSHAP,KernelSHAP,LeverageSHAP \
    --budget-mults 5,10,20,40,80,160 \
    --seeds 0,42,1337 --plot
```

sweeps $m \in \{5n, 10n, 20n, 40n, 80n, 160n\}$ over eleven games: the three SOUM synthetic games ($n = 6, 8, 10$) plus every dataset from the Musco & Witter paper (IRIS $n = 4$, California $n = 8$, Diabetes $n = 10$, Adult $n = 12$, Independent/Correlated $n = 60$, NHANES $n = 79$, Communities $n = 101$). Ground truth comes from `ExactComputer` for $n \leq 10$ and from `TreeExplainer` on the XGBoost model for the larger ones. All 4752 cells completed with `status="ok"`.

The metric reported below is the ℓ_2-norm error from the paper, $\|\tilde{\varphi} - \varphi\frac{\|}{\|}^2\varphi\|^2$, which is what Figures 3 and 4 in the paper also report. The plots below are lifted straight from `benchmark/results/sv_sweep_20260704_215230/plots/` lines are seed medians, shaded bands are the seed range.

1. Small–medium n: LeverageSHAP is a clear win in the sub-2^n regime

For $n \leq 12$, LeverageSHAP wins **34 out of 42** budget × game cells; Optimized Kernel SHAP wins 6 and vanilla Kernel SHAP wins 2. The wins are not marginal either. At $m = 10n$ on Diabetes ($n = 10$), the median ℓ_2 errors are

Method	**median $\|\tilde{\varphi} - \varphi\frac{\|}{\|}^2\varphi\|^2$ at $m = 10n$**
Kernel SHAP	$2.67 \cdot 10^{-2}$
Optimized Kernel SHAP	$4.76 \cdot 10^{-3}$
LeverageSHAP	$\mathbf{1.57 \cdot 10^{-3}}$ (roughly 3× better than OKS, 17× better than KS)

The Diabetes plot is the picture-perfect version of this story: all three methods decay with m, but the LeverageSHAP curve sits roughly one half-decade below Optimized Kernel SHAP until $m \geq 2^n = 1024$, at which point both collapse onto machine precision.

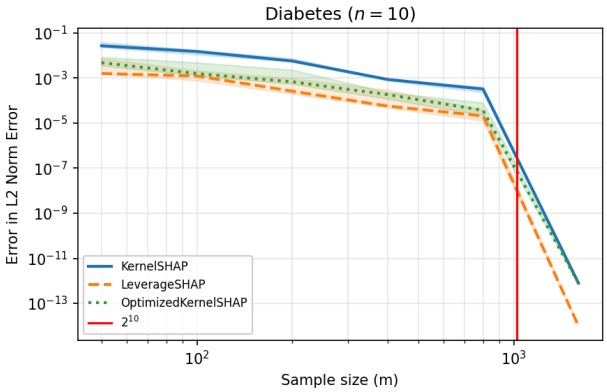

Figure 2: Diabetes ($n = 10$), ℓ_2 error.

California ($n = 8$) tells a slightly different story that lines up with prediction (2) above: because $2^n = 256$ is small, budgets $40, 80, 160$ already cover a big fraction of the support, and past $m = 320$ all three methods hit $\approx 10^{-15}$. In the sub-enumeration regime, LeverageSHAP still sits well below vanilla Kernel SHAP (10^{-2}–10^{-3} vs. 10^{-2}–10^{-1}), and is comparable to Optimized Kernel SHAP.

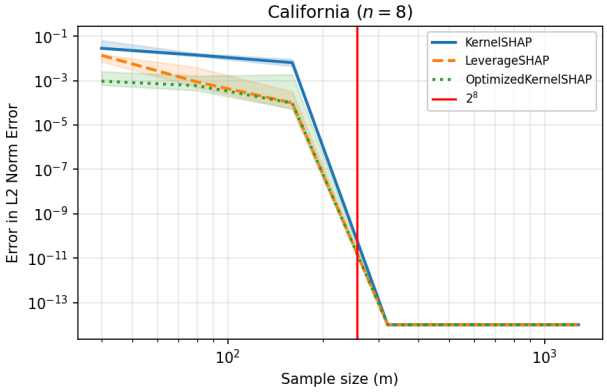

Figure 3: California ($n = 8$), ℓ_2 error.

2. Large n: LeverageSHAP wins more often, but the picture is noisier
For $n \geq 60$ (Independent, Correlated, NHANES, Communities), LeverageSHAP wins **13 of 24** cells against 8 for OKS and 3 for KS. The absolute gap between LeverageSHAP and OKS narrows sharply; often the two curves are within a factor of about 1.2 of each other, while both leave vanilla Kernel SHAP well behind at low m. This is exactly what the theory predicts. At large n, sampling *without replacement* is the dominant optimization, and the choice between kernel-weight sampling and leverage-score sampling becomes a second-order (but still visible) effect.

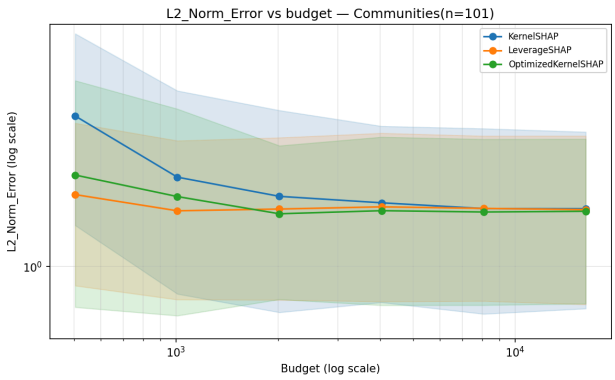

Figure 4: Communities ($n = 101$), ℓ_2 error.

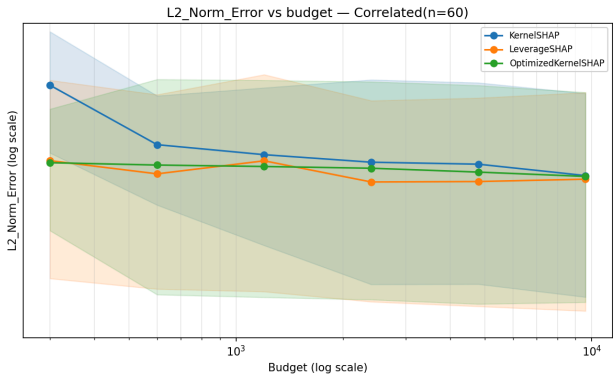

Figure 5: Correlated ($n = 60$), ℓ_2 error.

3. Where LeverageSHAP genuinely* loses*: high-γ datasets

Two datasets in the sweep, Adult ($n = 12$) and NHANES ($n = 79$), show LeverageSHAP *tied or slightly worse* than the Kernel SHAP variants. On Adult, $\|\tilde{\varphi} - \varphi\frac{\mathbb{1}^2}{\mathbb{1}}\varphi\|^2$ sits around **1.5–1.7 for all three methods and does not decrease with m**, which is a tell-tale sign that Corollary 3.2 is biting: the value function is far from the column span of A, γ is large, and the ℓ_2 metric is dominated by the irreducible term $\varepsilon\gamma\ \|\varphi\|^2$, which no regression-based estimator can escape.

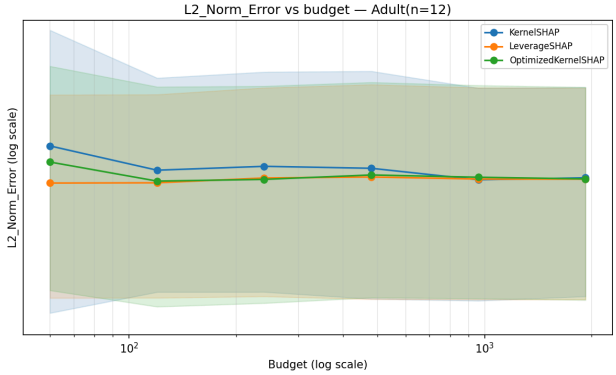

Figure 6: Adult ($n = 12$), the γ-limited plateau.

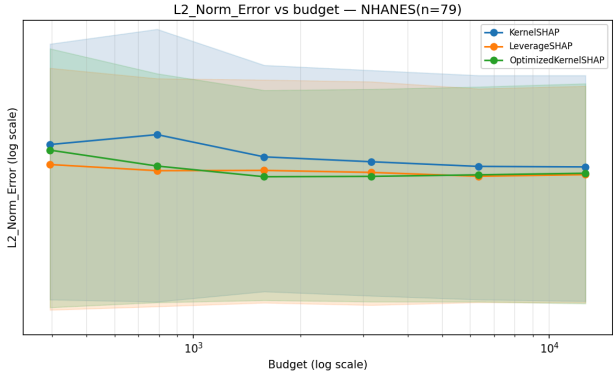

Figure 7: NHANES ($n = 79$), a milder version of the same effect.

The paper is upfront about this in Appendix G ("Exploring γ"): performance degrades with γ *for all three* regression-based methods, because they optimize the same objective. This is therefore not a defect of LeverageSHAP so much as a defect of the *reduction* to ℓ_2, one that the paper's own Theorem 1 (stated in the objective norm) never promises to fix.

4. Runtime: LeverageSHAP is also the fastest

A perhaps under-appreciated observation from the benchmark is that LeverageSHAP is consistently the *cheapest* of the three per call to `.approximate`. Median wall-clock across all cells:

Method	median runtime	mean runtime
Kernel SHAP	3.56 ms	34.6 ms
Optimized Kernel SHAP	2.79 ms	24.8 ms
LeverageSHAP	**1.25 ms**	**12.4 ms**

And the advantage widens with n:

n	KS median	OKS median	LeverageSHAP median
60	34.3 ms	26.3 ms	12.4 ms
79	52.9 ms	39.6 ms	22.6 ms
101	74.4 ms	55.7 ms	31.7 ms

Two things drive this. First, LeverageSHAP uses a *single* SVD-based weighted lstsq call in `leverageshap.py`, lines 129–134, whereas Optimized Kernel SHAP goes through the Lagrangian solve of Appendix I in the paper. Second, size-1 and size-$(n-1)$ coalitions, which Kernel SHAP oversamples heavily, are the most expensive to process, and leverage-score sampling deliberately gives them the same weight as every other size (see the sampling-distribution figure above).

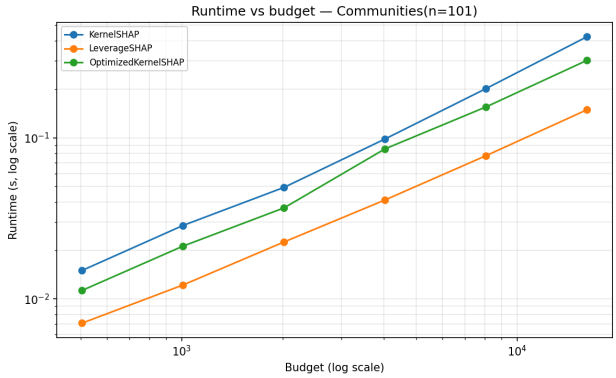

Figure 8: Runtime scaling on Communities ($n = 101$).

Summary of the win/loss picture

Regime	Prediction from paper	What the benchmark shows
Small n, $m \ll 2^n$	LeverageSHAP is best	✓ 34/42 wins for $n \leq 12$
Any n, $m \geq 2^n$	LeverageSHAP $\approx$ OKS, both at machine precision	✓ California/Diabetes at $m = 320+$ hit 10^{-15}; ordering is noise
Large n, moderate m	LeverageSHAP best, gap over OKS narrower	✓ 13/24 wins for $n \geq 60$; gap is about 1.2×, not 10×
High-γ value functions	All regression methods struggle on ℓ_2	✓ Adult and NHANES plateau for all three
Runtime	Not discussed in the paper	LeverageSHAP is fastest, 2–3× at large n

The one place where the benchmark deviates mildly from the story so far is Communities ($n = 101$) at very large m, where Optimized Kernel SHAP overtakes LeverageSHAP from $m = 4040$ onwards. The most plausible reading is that this is a *variance* phenomenon: with only 3 seeds and $n = 101$, both methods are already at their γ-limited floor, and the ordering within that floor is essentially random. The paper's own Figure 3 uses 100 seeds and reports IQR bands; with three seeds the shaded band on Communities is not informative, and no conclusion should be drawn from it.

Takeaway

LeverageSHAP behaves in practice the way its theory predicts. It wins where the regression-subsampling story is tight (moderate m, small–medium n), ties where full enumeration takes over ($m \geq 2^n$), stays close to Optimized Kernel SHAP where their design choices converge (large n with sampling-without-replacement doing the heavy lifting), and, most importantly, pays for its stronger guarantee with *no* runtime penalty. The one caveat is that on high-γ datasets neither LeverageSHAP nor Kernel SHAP can beat their ℓ_2 floor; the regression objective is the wrong yardstick there, and the paper says as much. Practically, the `leverageshap.py` implementation is a strict improvement to reach for whenever $m \ll 2^n$ and the game is not adversarial to the regression formulation.